

GESTÃO, ORQUESTRAÇÃO E AUTOMAÇÃO EM REDES OPENRAN

Provisionamento e gerenciamento de uma pilha Open RAN utilizando SMO

Relatório — Parte 2: implementação, operação e análise dos mecanismos

Integrantes: Charles Siqueira Xavier, Felipe Bartholo Favilla, Luciano Augusto Campagnoli da Silva, Thiago Bezerra Coelho

Setembro de 2026

Sumário

1. Introdução
2. Escopo e metodologia da implementação
3. Arquitetura implantada
4. Procedência dos artefatos e simplificações deliberadas
5. Provisionamento da pilha Open RAN
6. Mecanismos de gerenciamento e configuração
7. Mecanismos de coleta e barramento de dados
8. Monitoramento, telemetria e gerenciamento de falhas
9. Orquestração de políticas pela interface A1
10. Ciclo de vida de Network Functions
11. O portal de operação: arquitetura e decisões de projeto
12. Resultados da verificação end-to-end
13. Análise dos mecanismos frente ao modelo de referência de SMO
14. Guia de apoio ao uso da aplicação
15. Limitações conhecidas e trabalhos futuros
16. Conclusão
17. Referências

1. Introdução

A Parte 1 deste trabalho comparou as plataformas de Service Management and Orchestration disponíveis publicamente e selecionou o SMO da O-RAN Software Community — módulos OAM, responsável pela interface O1, e Non-RT RIC, responsável pela interface A1 — como a ferramenta a ser implementada. A escolha se apoiou em três pontos: alinhamento direto com as especificações da O-RAN Alliance, cobertura documentada de todos os itens do objeto de estudo e existência de um perfil leve, baseado em Docker Compose, compatível com os recursos de um laboratório acadêmico.

O relatório da Parte 1 registrou também a estratégia recomendada para esta segunda etapa: começar pelo perfil leve com escopo O1/OAM, validar a coleta de eventos e a configuração via NETCONF, e só então avaliar a inclusão do Non-RT RIC. Este relatório documenta o resultado da execução dessa estratégia — que foi cumprida integralmente, incluindo o Non-RT RIC.

O objetivo da Parte 2 é utilizar a ferramenta selecionada para realizar o provisionamento, o gerenciamento e o acompanhamento de uma pilha Open RAN, e analisar os mecanismos de gestão, orquestração e operação

empregados pela solução. O que se documenta aqui, portanto, não é apenas o que a plataforma promete, mas o que ela efetivamente faz quando colocada em operação — incluindo os pontos em que foi necessário divergir da distribuição oficial para que funcionasse.

Todo o material produzido está publicado em github.com/felipefavilla/smo-openran, com a pilha completa, o portal de operação, os scripts de verificação e este relatório.

2. Escopo e metodologia da implementação

2.1 O que foi implantado

A implantação reúne dez imagens de contêiner, todas artefatos oficiais publicados pela O-RAN Software Community e pelo ONAP, nas versões fixadas pela M-Release do projeto OAM (dezembro de 2025), mais um componente escrito por este grupo — um portal de operação. A distinção importa e é mantida em todo o relatório: tudo que executa funções de SMO é software oficial; o portal apenas consome as APIs desses componentes e as apresenta.

Camada	Serviço	Imagem oficial	Origem
SMO / OAM	controller — SDN-R (OpenDaylight)	<code>sdnr-image:13.0.1</code>	O-RAN SC
SMO / OAM	odlux — interface web do controlador	<code>sdnr-web-image:13.0.1</code>	O-RAN SC
SMO / OAM	ves-collector — ingestão de eventos O1	<code>vescollector:1.12.5</code>	ONAP
Mensageria	kafka + zookeeper	<code>strimzi/kafka:0.35.0-kafka-3.4.0</code>	Strimzi (versão fixada pela OSC)
Persistência	persistence — base do SDN-R	<code>mariadb:11.1.2</code>	versão fixada pela OSC
Rede gerenciada	o-du-1 — O-DU simulada	<code>nts-ng-o-ran-du:1.8.1</code>	O-RAN SC (sim-o1-interface)
Rede gerenciada	o-ru-1, o-ru-2 — O-RU fronthaul	<code>nts-ng-o-ran-ru-fh:1.8.1</code>	O-RAN SC (sim-o1-interface)
Non-RT RIC	a1-pms — A1 Policy Management Service	<code>nonrt-ric-a1-policy-management-service:2.3.1</code>	O-RAN SC
Non-RT RIC	near-rt-ric — simulador da interface A1	<code>a1-simulator:2.2.0</code>	O-RAN SC
Operação	smo-portal — portal deste trabalho	construída localmente	este trabalho

2.2 Método de trabalho

A implantação seguiu quatro etapas, cada uma validada antes da seguinte: infraestrutura (persistência e barramento de mensagens); núcleo OAM (coletor VES e controlador SDN-R); rede gerenciada (os três simuladores de funções de rede); e Non-RT RIC (serviço de políticas e simulador de Near-RT RIC). Cada etapa só foi considerada concluída com evidência de funcionamento — não a subida do contêiner, mas a resposta correta da API correspondente.

Essa disciplina revelou quatro divergências entre a distribuição oficial e o que de fato funciona no ambiente disponível. Todas estão documentadas na seção 4 e comentadas no ponto correspondente do arquivo de composição, porque são parte legítima do resultado da investigação: mostram onde a documentação oficial e o código publicado se descolam.

2.3 Ambiente de execução

Docker Desktop sobre Windows 11 com WSL2, 16 GB de memória física e 16 núcleos lógicos. A pilha completa consome cerca de 2,5 GB de memória em regime permanente, com o controlador OpenDaylight respondendo pela maior parte. O tempo de subida a frio é de aproximadamente cinco minutos, quase todo gasto no carregamento dos módulos Karaf do controlador.

3. Arquitetura implantada

A arquitetura reproduz, em escala reduzida, o modelo em camadas descrito na Parte 1: os elementos de rede na base, o núcleo OAM mediando a interface O1, o barramento de mensagens interligando produtores e consumidores de dados, o Non-RT RIC atuando sobre a interface A1, e uma camada de operação no topo.

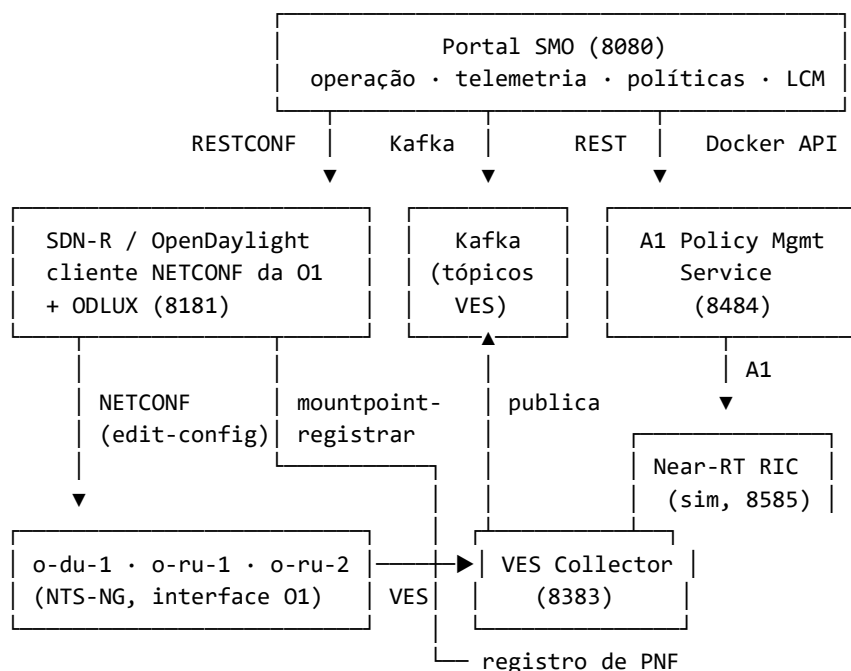


Figura 1 — Arquitetura da pilha implantada e os caminhos de dados entre seus componentes.

3.1 Como um elemento entra sob gestão do SMO

O caminho pelo qual uma função de rede passa a ser gerenciada é o mecanismo mais característico da solução da OSC, e vale descrevê-lo em detalhe porque ele integra, num único fluxo, três componentes distintos da plataforma:

1. A função de rede sobe e envia um evento VES de registro de PNF ao VES Collector, informando quem é, onde está e como pode ser alcançada.
2. O coletor valida o evento contra o esquema VES Common Event Format e o publica no tópico `unauthenticated.VES_PNFREG_OUTPUT` do barramento Kafka.
3. O `mountpoint-registrar`, componente do SDN-R, consome esse tópico e grava no datastore de configuração do controlador um nó NETCONF com o endereço, a porta e as credenciais do elemento.
4. O OpenDaylight abre a sessão NETCONF, lê as capacidades YANG que o elemento anuncia e publica o `mountpoint` correspondente. O elemento passa ao estado `connected`.
5. O portal detecta o novo elemento e aplica o perfil de provisionamento inicial pela interface O1 — descrito na seção 5 —, fazendo a telemetria começar a fluir.

O ponto a destacar é que nenhuma dessas etapas exige intervenção manual ou conhecimento prévio do elemento por parte do operador: o SMO descobre, monta e configura. É esse encadeamento que a especificação da interface O1 chama de gerenciamento de configuração, e ele funciona na distribuição da OSC como descrito.

4. Procedência dos artefatos e simplificações deliberadas

4.1 Procedência

Os arquivos de configuração do controlador, do coletor VES e da interface ODLUX foram copiados sem alteração de conteúdo funcional do repositório oficial `o-ran-sc/oam`, commit `bfdfa32`. O arquivo de configuração do A1 Policy Management Service vem do repositório `o-ran-sc/nonrtr-ic`. O único arquivo de configuração próprio é o que declara o Near-RT RIC desta implantação e os elementos que ele gerencia.

4.2 O que foi deliberadamente removido

A distribuição oficial sobe cerca de vinte contêineres. Os itens abaixo foram removidos por não pertencerem a nenhum ponto do objeto de estudo e por multiplicarem o custo de implantação sem contrapartida analítica. São decisões de escopo, registradas como tais:

Componente removido	Justificativa
Traefik (gateway TLS)	exige edição de <code>/etc/hosts</code> e emissão de certificados; os serviços são expostos diretamente em portas do host
Keycloak + PostgreSQL (OIDC)	o controlador roda com <code>ENABLE_OAUTH=false</code> e autenticação básica do OpenDaylight
Servidor DHCP em macvlan	endereçamento de laboratório; o Compose já fornece nomes e endereços estáveis
Node-RED, Jenkins, Wireshark	ferramentas auxiliares de desenvolvimento, fora do escopo
Grafana + InfluxDB	o portal cobre a visualização de telemetria com dois contêineres a menos
Clone dos esquemas 3GPP MnS no build do VES Collector	validam apenas eventos <code>stndDefined</code> 3GPP, que os simuladores não emitem; o Dockerfile oficial está preservado no repositório

Na mensageria, o Kafka roda com listener `PLAINTEXT` simples, sem SASL nem autorizador OPA, porque não há multi-tenancy nem exposição externa nesta implantação.

4.3 Divergências necessárias em relação ao compose oficial

Quatro ajustes não foram escolhas de escopo, mas condições para que a pilha funcionasse. Registrá-los é parte do resultado da investigação:

4.3.1 Verificação de saúde do controlador

O compose oficial verifica a saúde do SDN-R pelo endpoint `/ready`. Esse endpoint pertence à camada SDNC do ONAP e não existe na imagem quando ela roda em modo `SDNONLY` — ele responde 404 mesmo com o controlador plenamente operacional. A verificação foi trocada pelo próprio `RESTCONF` da topologia `NETCONF`, que é o que precisa responder para o SMO operar.

4.3.2 Conflito entre NETCONF Call Home e registro por VES

A distribuição oficial habilita simultaneamente dois caminhos de descoberta: o `NETCONF Call Home`, em que o elemento inicia a conexão com o controlador, e o registro por evento VES, em que o `mountpoint-registrar` cria o nó a partir do tópico de registro de PNF. Com os dois ativos, o mesmo dispositivo é montado duas vezes e o OpenDaylight falha com `"Mount point already exists"` e `ConflictingModificationAppliedException`, deixando o nó sem estado operacional — visível como um elemento montado porém sem status de conexão e sem capacidades listadas.

A implantação usa apenas o registro por VES, que é o fluxo canônico documentado pela OSC e o mais interessante do ponto de vista do objeto de estudo, por exercitar a cadeia completa `VES → Kafka → mountpoint-registrar → NETCONF`. A capacidade de `Call Home` do controlador permanece habilitada e a porta correspondente exposta.

4.3.3 Falha de segmentação no simulador de O-RU

No NTS-NG 1.8.1, a etapa `datastore-populate` do simulador de O-RU sofre falha de segmentação ao validar o módulo `o-ran-usermgmt`. O servidor `NETCONF` sobrevive, mas o daemon que faz o registro VES morre — o elemento fica acessível por `NETCONF` e invisível para o SMO. Desativada essa etapa, o elemento sobe com os dados que acompanham a imagem, determinísticos e suficientes para a interface O1. O O-DU mantém a etapa, na qual ela funciona.

4.3.4 Verificações de saúde sem curl

As imagens do A1 Policy Management Service e do ZooKeeper não incluem curl nem wget, de modo que uma verificação escrita com essas ferramentas nunca passa, ainda que o serviço esteja íntegro. As verificações foram reescritas com o /dev/tcp do bash.

5. Provisionamento da pilha Open RAN

A pilha gerenciada é composta por um O-DU e dois O-RU de fronthaul, todos simuladores oficiais do projeto simo1-interface da OSC. Esses simuladores implementam a interface O1 de verdade: servidor NETCONF com os modelos YANG das especificações O-RAN, emissão de eventos VES e suporte ao ciclo de vida de medições de desempenho.

5.1 Descoberta e montagem

Após a subida, os três elementos foram descobertos e montados automaticamente pelo mecanismo descrito na seção 3.1. O resultado observado, lido do RESTCONF do controlador:

Elemento	Estado da conexão	Módulos YANG	Capacidades de protocolo
o-du-1	connected	33	13
o-ru-1	connected	84	13
o-ru-2	connected	84	13

A lista anunciada mistura duas coisas, e vale separá-las: os módulos YANG que o elemento suporta e as capacidades do próprio protocolo NETCONF — candidate, validate, xpath e afins —, iguais em todos os elementos. São os módulos YANG que determinam quais caminhos de configuração são válidos para aquele elemento, e a diferença entre 33 e 84 reflete a diferença real entre o modelo de uma unidade distribuída e o de um rádio de fronthaul.

Essa contagem é a evidência direta do mecanismo de descoberta de capacidade da interface O1: o controlador lê, na negociação NETCONF, tudo o que o elemento declara suportar, sem que o operador precise informar nada sobre o equipamento.

5.2 Provisionamento inicial executado pelo SMO

Aqui a implementação encontrou um ponto que a documentação não antecipa. Os simuladores sobem com os parâmetros de telemetria vazios: conectam-se pela interface O1 mas não reportam absolutamente nada — sem período de heartbeat, sem registro periódico, sem geração de falhas. E o O-DU não produz nenhuma medição de desempenho até que exista um job de medição; o próprio modelo YANG declara que "para ativar a produção da medição especificada, o SMO precisa criar uma instância de performance-measurement-job".

Definir esses parâmetros é, por natureza, trabalho de gerenciamento de configuração do SMO — exatamente a função que a Parte 1 atribuiu ao módulo OAM. A implementação o faz pelo caminho normal da interface O1: um reconciliador no portal detecta cada elemento recém-conectado e aplica um perfil de dia 1, item a item, com releitura de confirmação após cada escrita.

Item do perfil	Caminho YANG	Efeito
Telemetria VES	nts-network-function:simulation/network-function/ves	registro de PNF, heartbeat de 30 s e reporte de falhas
Notificações NETCONF	.../network-function/netconf	notificações de falha pelo próprio canal NETCONF
Geração de falhas	.../network-function/fault-generation	cadência de emissão de alarmes, para exercitar o FM
Assinatura de streaming	o-ran-sc-du-hello-world:.../subscription-streams	destino VES das medições de desempenho (apenas O-DU)
Job de medição	o-ran-sc-du-hello-world:.../performance-measurement-jobs	ativa a produção de KPIs por célula (apenas O-DU)

O perfil é seletivo: cada item declara o módulo YANG de que depende, e o reconciliador aplica apenas os itens cujo módulo o elemento efetivamente anuncia. Por isso os O-RU recebem três itens e o O-DU, cinco. O job de medição é montado em tempo de execução a partir das células que o elemento expõe — descoberta antes de provisionamento, e não uma configuração fixa.

SMO Open RAN
O-RAN SC - M-Release

Visão geral

Inventário e topologia

Provisionamento O1

Falhas

Desempenho

Políticas A1

Ciclo de vida de NF

Ajuda

fluxo ao vivo

ODLUX ↗

VES Collector ↗

Provisionamento O1

Leitura e escrita de configuração nos elementos via RESTCONF/NETCONF

Atualizar

Perfil de provisionamento inicial

Aplicado automaticamente pelo SMO via interface O1 quando o elemento conecta

Os simuladores sobem com os parâmetros de telemetria vazios: conectam pela interface O1, mas não reportam nada. O SMO aplica este perfil em cada elemento assim que ele conecta — um `<edit-config>` por item, seguido de releitura de confirmação.

ITEM	CAMINHO YANG
Telemetria VES Registro de PNF, heartbeat periódico e reporte de falhas pela interface O1	nts-network-function:simulation/network-function/ves
Notificações NETCONF Notificações de falha pelo próprio canal NETCONF; Call Home desativado	nts-network-function:simulation/network-function/netconf
Padrão de geração de falhas Cadência com que o elemento emite eventos de falha, para exercitar o FM	nts-network-function:simulation/network-function/fault-generation
Assinatura de streaming Destino VES para onde o elemento envia as medições de desempenho	o-ran-sc-du-hello-world:network-function/subscription-streams=stream-smo
Job de medição de desempenho Ativa a produção de KPIs de vazão por célula, com granularidade de 30 s	o-ran-sc-du-hello-world:network-function/performance-measurement-jobs=pm-job-smo

APLICAÇÃO POR ELEMENTO

ELEMENTO	RESULTADO	ETAPAS	QUANDO	
o-du-1	✓ perfil aplicado	Telemetria VES - 204 Notificações NETCONF - 204 Padrão de geração de falhas - 204 Assinatura de streaming - 204 Job de medição de desempenho - 204	22:28:10	Reaplicar
o-ru-1	✓ perfil aplicado	Telemetria VES - 204 Notificações NETCONF - 204 Padrão de geração de falhas - 204	22:28:28	Reaplicar
o-ru-2	✓ perfil aplicado	Telemetria VES - 204 Notificações NETCONF - 204 Padrão de geração de falhas - 204	22:28:10	Reaplicar

Configuração via interface O1

Leitura e escrita no datastore do elemento, mediadas pelo SDN-R

HTTP 200

Elemento de rede

o-ru-1

Caminho YANG

nts-network-functionsimulation/network-function/ves

Datastore

config (running)

Caminhos sugeridos para este elemento

nts-network-function:simulation/network-function/ves
nts-network-function:simulation/network-function/netconf
nts-network-function:simulation/network-function/fault-generation
o-ran-usermgmt:users
ietf-interfaces:interfaces
ietf-hardware:hardware
o-ran-uplane-conf:user-plane-configuration
o-ran-sync:sync
o-ran-sc-du-hello-world:network-function
o-ran-sc-du-hello-world:network-function/subscription-streams=stream-smo

Sondados neste elemento: em destaque os que têm configuração gravada; esmaecidos, os ramos vazios; mais claros ainda, os módulos que o elemento não anuncia. Passe o cursor para ver o motivo.

Ler configuração

Aplicar configuração

Conteúdo (JSON conforme RFC 8040)

```
{
  "nts-network-function:ves": {
    "heartbeat-period": 30,
    "faults-enabled": true,
    "pnf-registration": true
  }
}
```

Leitura concluída. O SDN-R traduziu este GET RESTCONF em um `<get-config>` NETCONF para o-ru-1.

Figura 2 — Tela de provisionamento: perfil aplicado a cada elemento, com o código de resposta de cada etapa, e o editor de configuração por caminho YANG.

6. Mecanismos de gerenciamento e configuração

O mecanismo de configuração da plataforma é o modelo cliente-servidor NETCONF, exposto ao norte como RESTCONF. Na prática, um operador (ou uma aplicação) emite uma requisição HTTP ao controlador, e o controlador a traduz na operação NETCONF correspondente sobre o mountpoint do elemento:

Operação na interface norte (RESTCONF)

```
GET .../node=<id>/yang-
ext:mount/<caminho>?content=config
GET ...?content=nonconfig
PUT .../node=<id>/yang-ext:mount/<caminho>
DELETE .../node=<id>/yang-ext:mount/<caminho>
```

Operação emitida ao elemento (NETCONF)

```
get-config no datastore running
get no datastore operational
edit-config com substituição do ramo
edit-config com operação de remoção
```

Gestão, Orquestração e Automação em Redes OpenRAN — Parte 2 | Página 6 de 18

A verificação desse mecanismo foi feita com um teste de ida e volta: ler um ramo do datastore, alterar um valor, gravar e reler o mesmo caminho. Alterando o período de heartbeat de um O-RU de 30 para 20 segundos, o controlador respondeu HTTP 204 e a releitura devolveu o novo valor — confirmando que a escrita chegou ao elemento e foi aplicada, e não apenas aceita pelo controlador.

Vale registrar dois comportamentos observados que são próprios do RESTCONF e costumam confundir na primeira vez:

- um HTTP 409 com error-tag data-missing não significa que o módulo YANG seja desconhecido, mas que não há dado gravado naquele ramo específico do datastore — situação normal em ramos que o elemento nunca populou;
- a leitura de um nó de alto nível pode podar listas de estado que só aparecem quando se lê o nó filho diretamente. Isso foi observado com a lista de medições suportadas de uma célula, e obrigou o reconciliador a ler célula a célula em vez de fazer uma leitura única.

7. Mecanismos de coleta e barramento de dados

A coleta de dados operacionais segue um caminho único, independentemente do tipo de dado: o elemento emite um evento no formato VES Common Event Format por HTTP; o VES Collector valida o evento contra o esquema correspondente; e, aceito o evento, publica-o num tópico Kafka segregado pelo domínio do evento. Quem quiser consumir aquele tipo de dado assina o tópico correspondente.

A separação por tópico não é cosmética: é o que permite que consumidores diferentes — o mountpoint-registrar, uma rApp de garantia de serviço, um portal de operação — recebam exatamente o que lhes interessa, sem filtrar o fluxo inteiro. Os oito tópicos criados pelo coletor nesta implantação:

Tópico	Conteúdo	Consumidor nesta implantação
unauthenticated.VES_PNFREG_OUTPUT	registro de funções de rede	mountpoint-registrar do SDN-R e portal
unauthenticated.SEC_FAULT_OUTPUT	eventos de falha	portal (gerenciamento de falhas)
unauthenticated.SEC_HEARTBEAT_OUTPUT	sinais de vida periódicos	portal (supervisão)
unauthenticated.VES_O_RAN_SC_HELLO_WORLD_PM_STREAMING_OUTPUT	medições de desempenho em fluxo contínuo	portal (KPIs)
unauthenticated.VES_MEASUREMENT_OUTPUT	medições no formato VES clássico	portal
unauthenticated.VES_FILE_READY_OUTPUT	aviso de arquivo de medição disponível	—
unauthenticated.SEC_3GPP_FAULTSUPERVISION_OUTPUT	falhas no formato 3GPP	—
unauthenticated.SEC_OTHER_OUTPUT	demais eventos	portal

Uma observação relevante sobre a evolução da plataforma: a M-Release abandonou o DMaaP Message Router usado nas versões anteriores e passou a publicar diretamente no Kafka. Os arquivos de configuração do coletor ainda trazem URLs no formato antigo, mas apenas o nome do tópico é utilizado. Essa mudança tem consequência prática para quem escreve consumidores: o acesso passa a exigir um cliente Kafka, não mais chamadas REST.

8. Monitoramento, telemetria e gerenciamento de falhas

8.1 Gerenciamento de falhas

Os alarmes chegam ao SMO como eventos VES de domínio fault, contendo a condição de alarme, o problema específico, a severidade e a origem. O portal constrói a partir desse fluxo uma lista de alarmes correntes, agregando ocorrências repetidas da mesma condição no mesmo elemento e encerrando um alarme automaticamente quando o elemento reenvia a mesma condição com severidade NORMAL — que é a convenção de clear do formato VES.

Durante a validação, os simuladores geraram 174 eventos de falha, com severidades MAJOR e MINOR alternadas e condições realistas como "CPRI Port Down" e "Interface Fault". Um evento de falha crítica injetado manualmente no coletor percorreu toda a cadeia — coletor, Kafka, portal — e apareceu na tela de falhas em poucos segundos.

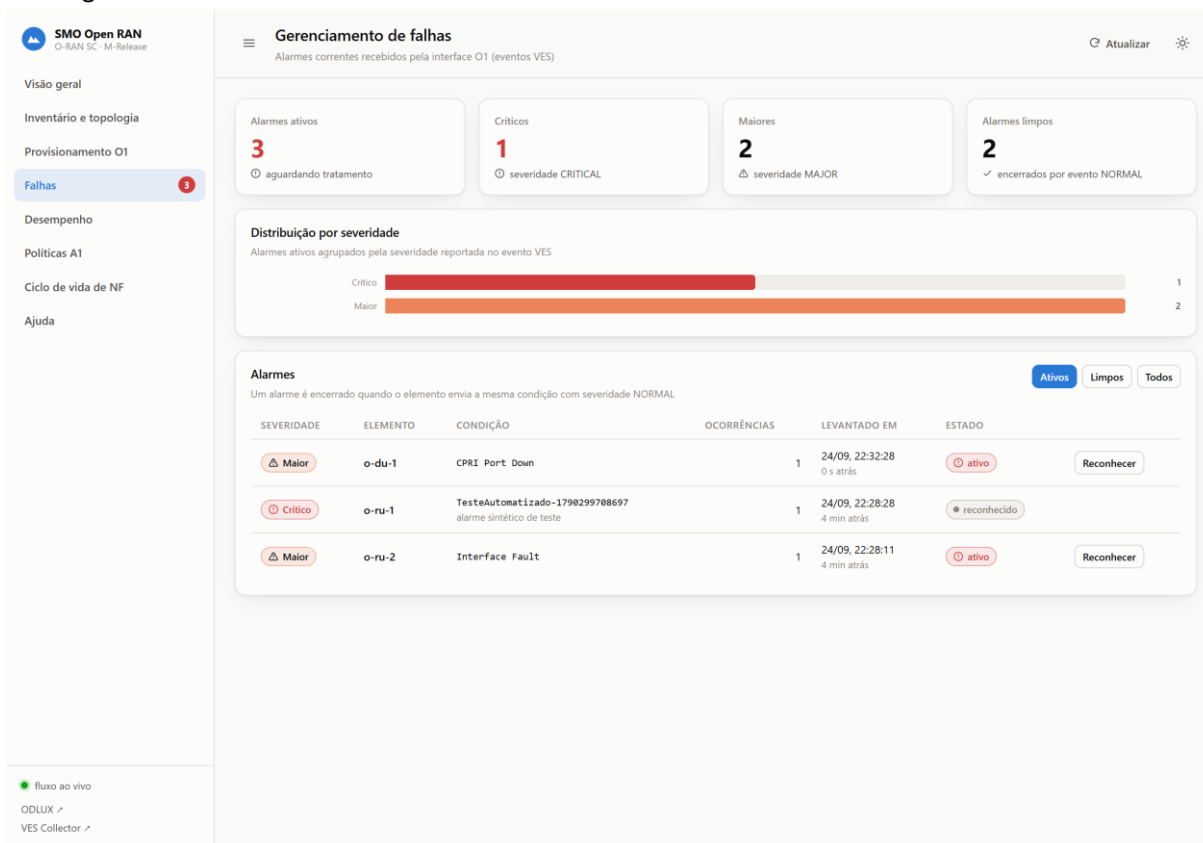


Figura 3 — Tela de gerenciamento de falhas: resumo por severidade, distribuição e lista de alarmes correntes com reconhecimento.

8.2 Telemetria de desempenho

A telemetria de desempenho exigiu o provisionamento descrito na seção 5.2. Uma vez ativo o job de medição, o O-DU passou a emitir, a cada 30 segundos, um evento de streaming com as medições de vazão média por célula, no sentido de subida e de descida, em kbit/s. Esses eventos chegam com domínio stdnDefined no cabeçalho VES e o namespace o-ran-sc-du-hello-world-pm-streaming-oas3, e carregam cada medição com sua referência completa de instância — o caminho YANG que identifica exatamente qual contador de qual célula está sendo reportado.

O portal converte essas referências em rótulos legíveis e mantém uma série temporal por indicador. Em uma hora de operação foram coletados 31 relatórios de medição e 271 heartbeats dos três elementos.

Desempenho e telemetria

Fluxo de eventos e indicadores coletados da rede

Atualizar

Vazão de eventos da interface O1

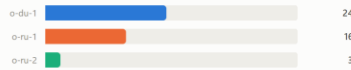
Eventos VES por minuto, empilhados por domínio

15 min 30 min 60 min



Atividade por função de rede

Total de eventos recebidos de cada elemento



Estado de reporte

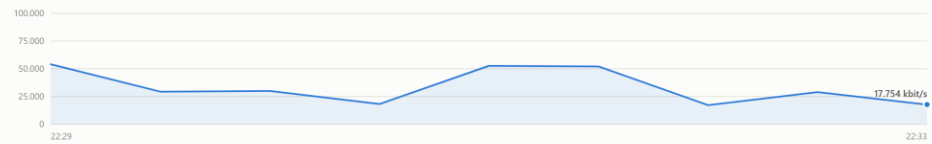
Último evento recebido de cada elemento

ELEMENTO	ÚLTIMO HEARTBEAT	EVENTOS
o-du-1	6 s atrás	24
o-ru-1	26 s atrás	16
o-ru-2	4 min atrás	3

User equipment average throughput downlink - cell-1

o-du-1 - 9 amostras

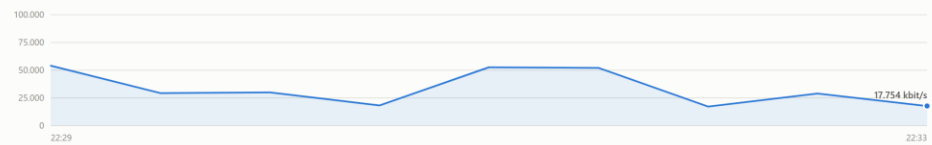
17.754 kbit/s



User equipment average throughput uplink - cell-1

o-du-1 - 9 amostras

17.754 kbit/s



Eventos recentes

Últimos eventos VES recebidos pelo barramento do SMO

HORÁRIO	DOMÍNIO	ELEMENTO	EVENTO
22:32:28	Falha	o-du-1	O_RAN_COMPONENT_Alarms
22:32:26	Heartbeat	o-du-1	Controller
22:33:00	PM streaming	o-du-1	stndDefined_performanceMeasurementStreaming
22:32:06	Heartbeat	o-ru-1	Controller
22:31:56	Heartbeat	o-du-1	Controller
22:32:30	PM streaming	o-du-1	stndDefined_performanceMeasurementStreaming
22:31:44	Falha	o-du-1	O_RAN_COMPONENT_Alarms
22:31:36	Heartbeat	o-ru-1	Controller
22:31:36	Falha	o-ru-1	O_RAN_COMPONENT_Alarms
22:31:26	Heartbeat	o-du-1	Controller
22:32:00	PM streaming	o-du-1	stndDefined_performanceMeasurementStreaming
22:31:14	Falha	o-du-1	O_RAN_COMPONENT_Alarms
22:31:07	Heartbeat	o-ru-1	Controller
22:30:56	Heartbeat	o-du-1	Controller
22:30:51	Falha	o-ru-1	O_RAN_COMPONENT_Alarms
22:31:30	PM streaming	o-du-1	stndDefined_performanceMeasurementStreaming
22:30:37	Heartbeat	o-ru-1	Controller
22:30:26	Heartbeat	o-du-1	Controller
22:30:21	Falha	o-ru-1	O_RAN_COMPONENT_Alarms
22:31:00	PM streaming	o-du-1	stndDefined_performanceMeasurementStreaming
22:30:14	Falha	o-du-1	O_RAN_COMPONENT_Alarms
22:30:07	Heartbeat	o-ru-1	Controller
22:29:57	Heartbeat	o-du-1	Controller
22:30:30	PM streaming	o-du-1	stndDefined_performanceMeasurementStreaming
22:29:37	Heartbeat	o-ru-1	Controller
22:29:29	Falha	o-du-1	O_RAN_COMPONENT_Alarms

Figura 4 — Tela de desempenho: vazão de eventos por domínio, distribuição por elemento e séries temporais dos indicadores coletados.

9. Orquestração de políticas pela interface A1

O Non-RT RIC foi implantado com dois componentes: o A1 Policy Management Service, que concentra a comunicação com os Near-RT RIC, e o simulador de Near-RT RIC mantido pela própria OSC, que implementa o lado sul da interface A1.

O serviço de políticas descobre os RICs a partir de sua configuração, consulta periodicamente quais tipos de política cada um suporta e mantém o ciclo de vida das instâncias. Um detalhe de operação que não é evidente na documentação: o simulador sobe sem nenhum tipo de política definido, de modo que o Non-RT RIC não tem o que instanciar até que um tipo seja carregado no RIC. O portal expõe essa carga como uma ação explícita, o que torna o encadeamento visível ao operador.

O ciclo completo foi verificado:

1. carga de um tipo de política de alvo de desempenho por célula no Near-RT RIC — o simulador exige o tipo embrulhado em uma estrutura com nome, descrição, identificador e esquema de criação, e não o esquema JSON isolado;
2. sincronização automática do Non-RT RIC, que passa a listar o tipo poucos segundos depois;
3. criação de uma instância de política pelo A1 Policy Management Service, com resposta HTTP 201;
4. confirmação no próprio Near-RT RIC de que a política foi entregue pela interface A1 — evidência de que o SMO não apenas registrou a política, mas a transmitiu ao RIC;
5. consulta do estado de aplicação reportado pelo RIC e remoção da instância.

Figura 5 — Tela de políticas A1: RIC gerenciado, tipos disponíveis, formulário de criação e instâncias com o estado reportado pelo próprio RIC.

10. Ciclo de vida de Network Functions

A Parte 1 registrou que, no perfil leve da plataforma, o ciclo de vida de funções de rede se resume a iniciar, monitorar e encerrar os contêineres que as implementam, e que no perfil completo ele passaria a ser gerido por Helm sobre Kubernetes — e, sobre O-Cloud, pela função O2dms. A implementação confirma essa leitura: não há na distribuição da OSC um módulo de orquestração de funções de rede que atue sobre o perfil Docker Compose. O portal exerce esse ciclo de vida pela API da Docker Engine, restrito às funções de rede gerenciadas — a plataforma em si aparece apenas para diagnóstico, sem ações, para que o portal não desligue a infraestrutura que o sustenta.

O ponto analiticamente interessante é o acoplamento entre o ciclo de vida e a interface O1, que a implementação torna observável: ao parar um O-RU, o mountpoint NETCONF correspondente deixa o estado connected em poucos segundos, a aresta O1 daquele elemento muda de cor no diagrama de topologia e o fluxo de eventos daquele elemento cessa. Ao reiniciá-lo, o ciclo completo de registro por VES e remontagem se repete sem qualquer ação manual. É a demonstração mais direta de que o SMO mantém um modelo vivo da rede, e não um inventário estático.

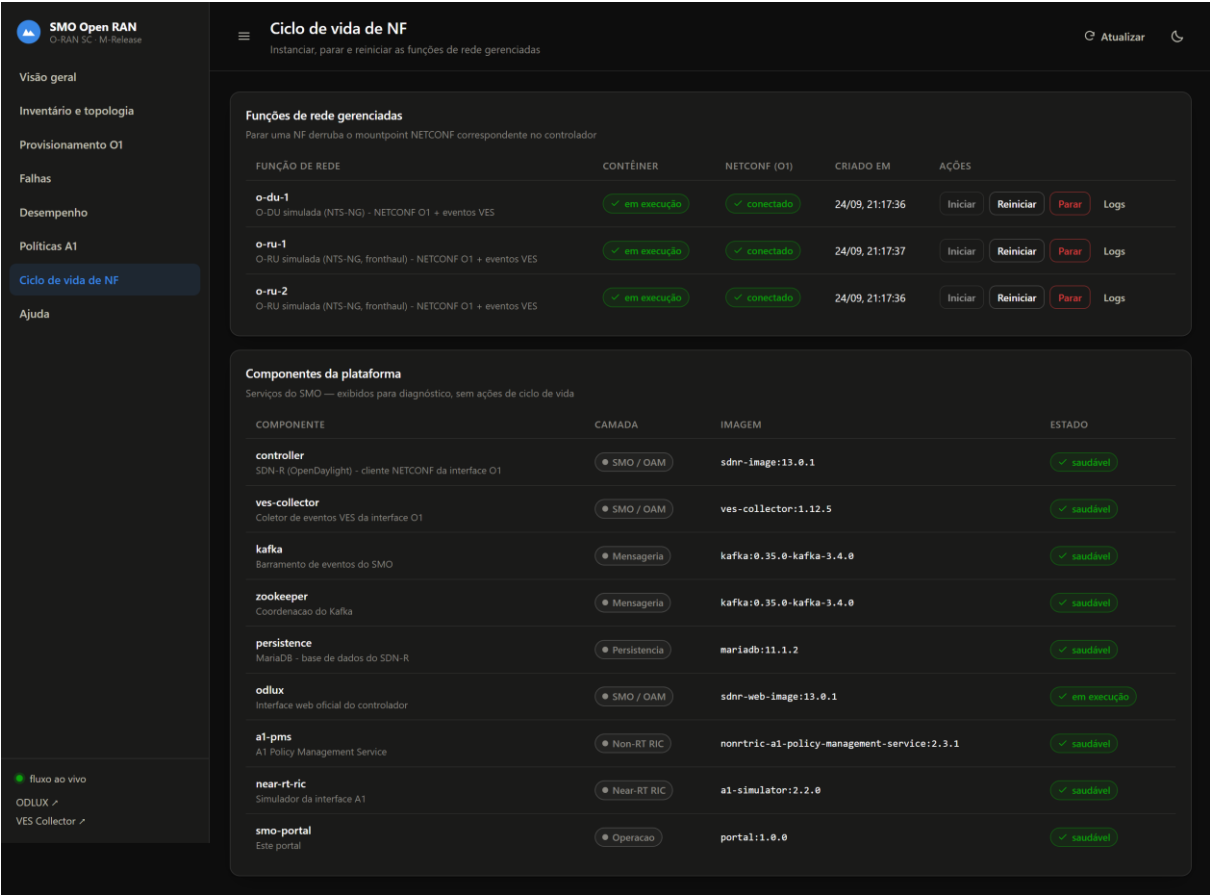


Figura 6 — Tela de ciclo de vida: estado de cada função de rede, estado do mountpoint O1 correspondente e ações de início, parada e reinício.

11. O portal de operação: arquitetura e decisões de projeto

O portal foi construído sob uma restrição autoimposta: não reimplementar nenhuma função de SMO. Ele lê e escreve através das APIs dos componentes oficiais e apresenta o resultado. Essa disciplina mantém a análise honesta — o que aparece nas telas é o que a plataforma da OSC efetivamente entrega, não uma simulação construída por cima dela.

Tela	Fonte de dados	Item do objeto de estudo evidenciado
Visão geral	agregação de todas as fontes	arquitetura e serviços de SMO em operação
Inventário e topologia	RESTCONF topology-netconf	componentes O-RAN e descoberta de capacidade

Provisionamento O1	RESTCONF sobre yang-ext:mount	gerenciamento de configuração
Falhas	tópico VES de falhas	gerenciamento de falhas
Desempenho	tópicos VES de medição e heartbeat	monitoramento e telemetria
Políticas A1	A1 PMS e Near-RT RIC	Non-RT RIC e orquestração de políticas
Ciclo de vida de NF	Docker Engine API	ciclo de vida de Network Functions
Ajuda	conteúdo estático	apoio ao uso, embutido na aplicação

11.1 Decisões técnicas

O backend é Node.js com duas dependências apenas — um servidor HTTP e um cliente Kafka. O frontend não tem etapa de compilação nem dependência de rede: são módulos JavaScript nativos, folha de estilo própria e gráficos desenhados em SVG. A aplicação inteira funciona sem acesso à internet, o que importa num laboratório fechado e reduz a superfície de manutenção do trabalho.

A paleta de séries dos gráficos foi validada quanto à separação sob daltonismo e ao contraste contra o fundo, nos modos claro e escuro. Cores de estado nunca carregam significado sozinhas: acompanham sempre ícone e rótulo textual. O fluxo de eventos chega ao navegador por Server-Sent Events, sem consulta periódica pelo cliente.

Não há banco de dados no portal: os dados persistentes vivem nos componentes oficiais — o inventário no SDN-R com sua base MariaDB, o histórico de eventos no Kafka. O portal mantém apenas uma janela deslizante em memória, o que significa que seus contadores reiniciam quando o contêiner reinicia. É uma limitação consciente, coerente com o papel de camada de apresentação.

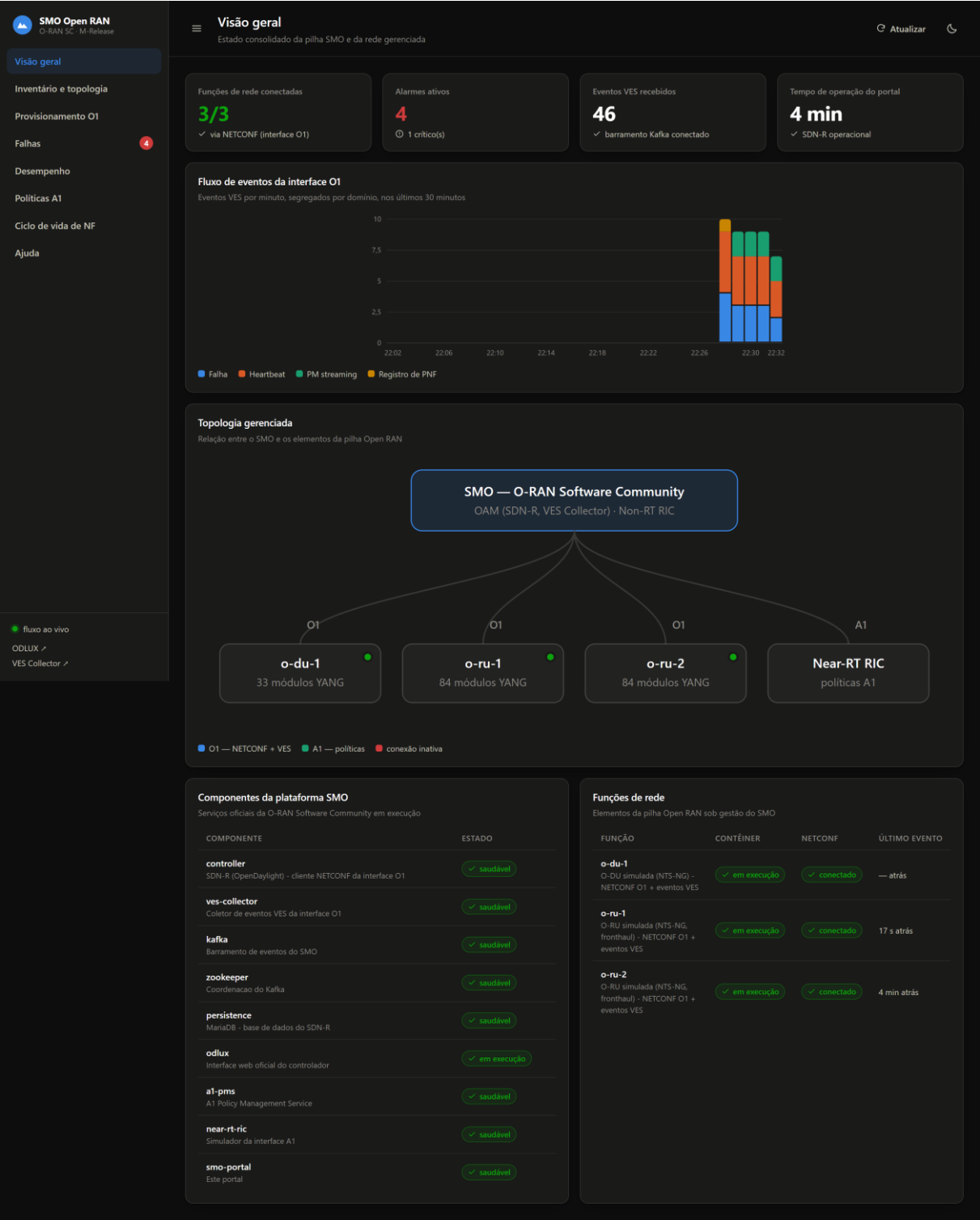


Figura 7 — Visão geral no tema escuro: indicadores, fluxo de eventos por domínio, topologia gerenciada e saúde dos componentes.

12. Resultados da verificação end-to-end

O repositório inclui um roteiro de verificação executável que exercita os sete mecanismos pedidos pelo objeto de estudo e imprime o resultado de cada passo. Ele serve tanto como teste de regressão quanto como roteiro de demonstração. Resultado da última execução:

Passo verificado	Resultado
Saúde de todos os serviços da pilha	nenhum serviço em estado unhealthy
O1 — descoberta de elementos	3 elementos montados, com 33 e 84 módulos YANG
O1 — leitura de configuração	HTTP 200 no mountpoint NETCONF

O1 — escrita de configuração	HTTP 204, confirmado por releitura do datastore
O1 — tópicos de telemetria	8 tópicos VES criados pelo coletor
O1 — consumo pelo portal	eventos contabilizados por domínio
FM — aceitação do evento de falha	HTTP 202 no VES Collector
FM — propagação do alarme	alarme crítico visível no portal
A1 — criação de política	HTTP 201 no Non-RT RIC
A1 — entrega ao RIC	política confirmada no próprio Near-RT RIC
A1 — remoção	HTTP 204
LCM — parada de uma NF	mountpoint O1 correspondente deixou o estado connected
LCM — reinício da NF	novo registro de PNF e remontagem automática

Treze passos concluídos, nenhuma falha. Os números observados em cerca de uma hora de operação contínua: 10 eventos de registro de PNF, 271 heartbeats, 174 eventos de falha e 31 relatórios de medição de desempenho.

12.1 Suíte de testes do portal

Além do roteiro de demonstração, o repositório traz uma suíte de 87 testes automatizados sobre o portal: 60 sobre a API e 27 sobre a interface, estes últimos executados num navegador real. Nenhum deles usa simulação de dependência — cada caso percorre o caminho completo até o componente oficial correspondente, de modo que uma falha indica divergência real entre o portal e a pilha, e não um dublê desatualizado.

A suíte encontrou dois defeitos que o uso manual não havia revelado. O mais relevante: a tela de políticas nunca listou tipos nem instâncias, porque o cliente lia as chaves "policy_type_ids" e "policy_ids" enquanto o A1 Policy Management Service devolve "policytype_ids" e um arranjo "policies" com as instâncias completas. O segundo: a carga do tipo de política falhava com HTTP 400 quando o tipo já existia com instâncias associadas, porque o simulador recusa redefini-lo nessa situação.

O episódio é ilustrativo de um risco desta classe de integração. Ambos os defeitos estavam em nomes de campo e em condições de borda de APIs de terceiros — exatamente o tipo de erro que passa despercebido numa inspeção visual, porque a tela simplesmente mostra uma lista vazia, indistinguível de um estado legítimo sem dados.

13. Análise dos mecanismos frente ao modelo de referência de SMO

A Parte 1 avaliou a plataforma escolhida contra os sete componentes canônicos do modelo de referência de SMO, a partir da documentação. Esta seção repete a avaliação, agora com evidência de execução — o que muda o peso de algumas conclusões.

Componente	Avaliação na Parte 1	Observado na implementação
OAM	implementado	confirmado. Configuração, coleta e falhas funcionam pela O1, com descoberta automática de elementos. É o núcleo maduro da plataforma.
Non-RT RIC	implementado	confirmado no que toca a políticas A1. O ciclo completo de tipo, instância e entrega ao RIC funciona. O rApp Manager, porém, exige o perfil Kubernetes e ficou fora desta implantação.
TE&IV	implementado	parcialmente observado. A topologia de mountpoints e as capacidades YANG estão disponíveis pelo controlador, mas o projeto TEIV dedicado não integra o perfil Docker Compose.
SA	parcial	confirmado como parcial. Os dados que alimentariam a garantia de serviço estão todos disponíveis; o fechamento da malha depende de uma rApp que não vem pronta.
NFO	parcial	confirmado como parcial, e a implementação esclarece por quê: no perfil leve não há orquestrador de funções de rede, apenas o ciclo de vida dos contêineres. O acoplamento com a O1 é real e observável, mas quem o exerce é o operador.
SO	não implementado	não observado. Permanece fora do escopo da distribuição.

FOCOM	não implementado	não observado. Depende do O-Cloud, que exige StarlingX em hardware dedicado.
-------	------------------	--

A implementação não contradiz a análise da Parte 1, mas acrescenta uma nuance que a leitura da documentação não permitia antecipar: a distância entre "a plataforma implementa a função" e "a função está operacional após a instalação" é maior do que a documentação sugere. Três exemplos concretos surgiram nesta implantação — a telemetria que não flui até que o SMO a configure, o tipo de política que precisa ser carregado no RIC antes de qualquer instância, e o job de medição sem o qual não há KPI algum.

Essa distância não é um defeito da plataforma: é a natureza de um SMO. O que a OSC entrega são os mecanismos; a configuração que os coloca em operação é responsabilidade de quem opera a rede. O que a implementação mostra é que essa responsabilidade é maior do que a documentação deixa entrever, e que boa parte do trabalho de colocar um SMO em produção está justamente aí.

14. Guia de apoio ao uso da aplicação

14.1 Pré-requisitos

- Docker Engine 24 ou superior, com Docker Compose v2;
- 8 GB de memória disponíveis ao Docker para a pilha completa (6 GB sem o Non-RT RIC);
- cerca de 12 GB de espaço em disco para as imagens;
- portas livres no host: 8080, 8181, 8282, 8383, 8484, 8585 e 4334.

No Docker Desktop com WSL2, caso a memória padrão seja insuficiente, crie o arquivo `%UserProfile%\wslconfig` com uma seção `[wsl2]` e a linha `memory=10GB`, e execute `wsl --shutdown` antes de subir a pilha.

14.2 Instalação

```
git clone https://github.com/felipefavilla/smo-openran.git
cd smo-openran/deploy
```

```
# núcleo O1 + pilha Open RAN simulada + portal
./scripts/up.sh
```

```
# ou, incluindo o Non-RT RIC (interface A1)
./scripts/up.sh --with-a1
```

O script constrói as duas imagens locais, sobe a pilha na ordem correta e aguarda cada camada ficar saudável, com indicação de progresso. A primeira execução baixa cerca de 5 GB de imagens. O controlador OpenDaylight leva de três a cinco minutos para ficar saudável na primeira subida — é o passo mais demorado, e é normal. Para encerrar: `./scripts/down.sh` preserva os volumes; `./scripts/down.sh --purge` remove também os dados.

14.3 Primeiro acesso

O portal fica em `http://localhost:8080` e não exige autenticação. A interface oficial do controlador, o ODLUX, fica em `http://localhost:8181`, com usuário e senha admin.

Nos primeiros minutos após a subida, a sequência esperada é: os elementos aparecem em Inventário com estado conectado; em seguida o portal aplica o perfil de provisionamento inicial, visível na tela de Provisionamento; e então começam a chegar heartbeats, falhas e medições, visíveis em Desempenho e Falhas. Se após cinco minutos a tela de Inventário continuar vazia, consulte a seção de diagnóstico.

14.4 As telas

- **Visão geral** — estado consolidado: funções de rede conectadas, alarmes ativos, eventos recebidos e fluxo por minuto separado por domínio. O diagrama de topologia mostra cada aresta O1 com a cor do estado real da conexão.

- **Inventário e topologia** — um cartão por elemento, com endereço, estado da conexão e a lista completa de módulos YANG anunciados. O campo de filtro ajuda a localizar um módulo entre centenas.
- **Provisionamento O1** — onde a configuração é efetivamente aplicada. Escolha o elemento, informe um caminho YANG ou use um dos atalhos, clique em Ler configuração, edite o JSON e clique em Aplicar configuração. A releitura exibida abaixo comprova que o elemento aplicou a mudança. Os atalhos são sondados no próprio elemento quando a tela abre e ordenados pelo resultado — em destaque os que têm configuração gravada, esmaecidos os ramos vazios, e mais claros ainda os módulos que o elemento não anuncia.
- **Falhas** — alarmes correntes. Um alarme é encerrado automaticamente quando o elemento reenvia a mesma condição com severidade NORMAL. O botão Reconhecer marca o alarme como tratado sem removê-lo.
- **Desempenho** — vazão de eventos por minuto, distribuição por elemento, séries temporais dos indicadores e lista dos eventos mais recentes. Passe o cursor sobre qualquer gráfico para ver os valores exatos.
- **Políticas A1** — disponível apenas com o perfil A1 ativo. O primeiro passo é carregar o tipo de política no RIC; em seguida é possível criar instâncias e acompanhar o estado reportado pelo próprio RIC.
- **Ciclo de vida de NF** — iniciar, parar e reiniciar funções de rede, e inspecionar os registros de qualquer contêiner. Pare uma função de rede e observe a coluna NETCONF mudar para desconectado.
- **Ajuda** — este mesmo guia, embutido na aplicação, sempre acessível durante a operação.

14.5 Roteiro de demonstração

Sequência sugerida para apresentar a solução em funcionamento:

1. Abrir a Visão geral e confirmar as funções de rede conectadas e o fluxo de eventos ativo.
2. Em Inventário, mostrar os módulos YANG anunciados por um O-RU — a descoberta de capacidade da interface O1.
3. Em Provisionamento O1, ler um caminho, alterar um valor, aplicar e mostrar a releitura confirmando a gravação.
4. Em Ciclo de vida, parar um O-RU e voltar à Visão geral: a aresta O1 correspondente fica vermelha no diagrama.
5. Reiniciar o O-RU e acompanhar o novo registro de PNF chegando em Desempenho.
6. Em Políticas A1, carregar o tipo, criar uma política e mostrar o estado confirmado pelo Near-RT RIC.

O script `deploy/scripts/demo.sh` executa essa mesma sequência pela linha de comando e imprime o resultado de cada passo; `deploy/scripts/status.sh` mostra o estado dos contêineres, dos mountpoints NETCONF e dos tópicos VES.

14.6 Diagnóstico

Sintoma	O que verificar
Nenhum elemento em Inventário	Confirme que o controlador está saudável. O ciclo completo — registro de PNF, criação do mountpoint e sessão NETCONF — leva de um a dois minutos após cada simulador subir. O script <code>status.sh</code> mostra o estado de cada mountpoint.
Nenhum evento em Desempenho	O indicador no rodapé do menu lateral mostra o estado do fluxo ao vivo. Verifique os registros do Kafka e do coletor VES. Os contadores do portal vivem em memória e zeram a cada reinício do contêiner.
Contadores do portal parados	Só pode haver uma instância do portal por vez: uma segunda instância apontando ao mesmo barramento divide as partições do Kafka e consome mensagens da primeira.
Tela de Políticas A1 indisponível	O perfil A1 é opcional. Suba-o com <code>docker compose --profile a1 up -d</code> .
Leitura devolve HTTP 409 data-missing	O elemento anuncia o módulo YANG, mas não há dado gravado naquele ramo. É esperado em ramos nunca populados, e a sondagem dos atalhos já sinaliza

O portal não reflete uma mudança recente no código	esse caso antes do clique. Um HTTP 400 indica módulo que o elemento não anuncia.
Carregar o tipo de política devolve HTTP 400	Os módulos são servidos com revalidação obrigatória e o portal recarrega sozinho ao detectar um build novo. Se a tela ainda parecer antiga, force a recarga com Ctrl+Shift+R.
Aviso TimeoutNegativeWarning nos registros do portal	O simulador recusa redefinir um tipo que já tenha instâncias de política. O botão verifica antes de gravar e informa quando o tipo já está disponível; a recusa só aparece se houver instâncias a remover.
Erro de configuração nos registros do coletor VES	Vem da biblioteca cliente do Kafka, não do código do portal. O Node limita o valor e o consumo segue normal.
	O coletor tenta buscar configuração dinâmica num serviço do ONAP que não existe nesta implantação e recai na configuração estática montada. Ocorre também na implantação oficial e é inofensivo.

15. Limitações conhecidas e trabalhos futuros

- **Interface O2 e O-Cloud.** Permanecem fora do escopo, como a Parte 1 já antecipava. A aplicação O2 do StarlingX exige instalação em hardware dedicado, inviável no ambiente disponível. Simular a interface enfraqueceria a análise em vez de fortalecê-la.
- **rApp Manager.** O gerenciamento de ciclo de vida de rApps do Non-RT RIC depende de Helm sobre Kubernetes e não integra o perfil Docker Compose. A interface A1 foi exercitada diretamente pelo serviço de políticas.
- **Pilha de RAN real.** A rede gerenciada é composta por simuladores oficiais, que implementam a interface O1 fielmente mas não processam sinal de rádio. A integração com srsRAN ou OpenAirInterface tem caminho documentado e seria a evolução natural.
- **Persistência da telemetria.** O portal mantém uma janela em memória. Uma evolução direta seria escrever as séries num banco de séries temporais, aproximando-o do perfil oficial com InfluxDB.
- **Segurança.** A implantação roda sem TLS e sem OIDC, por opção de escopo. Uma implantação de produção reincorporaria o gateway e o servidor de identidade que foram removidos.

16. Conclusão

A ferramenta selecionada na Parte 1 foi implantada e utilizada para provisionar, gerenciar e acompanhar uma pilha Open RAN, cumprindo o objetivo da Parte 2. Os sete itens do objeto de estudo foram exercitados sobre componentes oficiais da O-RAN Software Community, com evidência de execução para cada um.

Sobre os mecanismos empregados pela solução, três conclusões se destacam. A primeira é que o núcleo de gerência da RAN da OSC é maduro e funciona como a especificação descreve: a cadeia que vai do registro por evento VES até a configuração por NETCONF opera sem intervenção manual, e o modelo de rede que o SMO mantém reage em segundos a mudanças no estado dos elementos.

A segunda é que a fronteira entre o que a plataforma implementa e o que o operador precisa configurar está mais adiante do que a documentação sugere. Elementos conectam-se sem reportar nada até que o SMO os configure; não há KPI antes que um job de medição seja criado; não há política antes que um tipo seja carregado no RIC. Reconhecer essa fronteira é, em boa medida, entender o que significa operar um SMO.

A terceira é que o custo de colocar a plataforma em funcionamento está mais na integração do que nos componentes. Nenhum dos quatro problemas encontrados era um defeito funcional de um módulo: eram uma verificação de saúde apontando para um endpoint inexistente, dois mecanismos de descoberta concorrendo entre si, uma etapa opcional de um simulador com falha de memória, e imagens sem as ferramentas que suas próprias verificações presumiam. São problemas de montagem, e é neles que o tempo de uma implantação real se concentra.

O resultado está publicado com tudo o que é necessário para reproduzi-lo: a pilha completa, o portal de operação, os scripts de verificação e este relatório.

17. Referências

O-RAN SOFTWARE COMMUNITY. OAM — SMO Deployment and Configuration. Disponível em: <https://docs.o-ran-sc.org/projects/o-ran-sc-oam/en/latest/config-deploy.html>. Acesso em: set. 2026.

O-RAN SOFTWARE COMMUNITY. Repositório oam (M-Release). Disponível em: <https://github.com/o-ran-sc/oam>. Acesso em: set. 2026.

O-RAN SOFTWARE COMMUNITY. Repositório sim-o1-interface — NTS Network Function. Disponível em: <https://github.com/o-ran-sc/sim-o1-interface>. Acesso em: set. 2026.

O-RAN SOFTWARE COMMUNITY. Non-RT RIC — A1 Policy Management Service. Disponível em: <https://github.com/o-ran-sc/nonrt-ric>. Acesso em: set. 2026.

O-RAN SOFTWARE COMMUNITY. Documentação oficial dos projetos. Disponível em: <https://docs.o-ran-sc.org/>. Acesso em: set. 2026.

O-RAN ALLIANCE. O-RAN Architecture Description. Disponível em: <https://www.o-ran.org/>. Acesso em: set. 2026.

BJORKLUND, M. (Ed.). RFC 6241 — Network Configuration Protocol (NETCONF). IETF, 2011.

BIERMAN, A.; BJORKLUND, M.; WATSEN, K. RFC 8040 — RESTCONF Protocol. IETF, 2017.

ONAP. VES Collector — DCAE Gen2. Disponível em: <https://docs.onap.org/>. Acesso em: set. 2026.

3GPP. TS 28.552 — Management and orchestration; 5G performance measurements.

FAVILLA, F. B. et al. Relatório — Parte 1: pesquisa, comparação e seleção de plataforma de SMO. Especialização Open RAN / CPQD, set. 2026.